LAPORAN PRAKTIK
PENGGUNAAN GIT DAN GITHUB DALAM PENGELOLAAN PROYEK PEMROGRAMAN WEB



Disusun oleh:

Muhammad Sahal Anwar Hadi (24260032)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul "Laporan Praktik Penggunaan Git dan GitHub dalam Pengelolaan Proyek Pemrograman Web". Laporan ini disusun sebagai tugas pengganti mata kuliah Pengolahan Citra Digital.

Laporan ini disusun berdasarkan kegiatan praktik secara langsung, yaitu pengelolaan sebuah proyek website sederhana bernama HelloWorld menggunakan Git sebagai sistem kontrol versi dan GitHub sebagai layanan penyimpanan repository jarak jauh. Kegiatan praktik mencakup instalasi dan konfigurasi Git, inisialisasi repository, pembuatan commit, pengunggahan proyek ke GitHub, pembaruan file, serta penggunaan branch dan merge.

Penulis menyampaikan terima kasih kepada Bapak Andi Aljabar, S.Kom., M.T.I. selaku dosen pengampu yang telah memberikan kesempatan, arahan, dan bimbingan dalam pelaksanaan tugas ini. Penulis juga menyampaikan terima kasih kepada semua pihak yang telah membantu selama proses praktikum dan penyusunan laporan.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca, khususnya mahasiswa yang ingin mempelajari pengelolaan proyek perangkat lunak menggunakan Git dan GitHub.

Bogor, 2 November 2026

Penulis,

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **3**
1.1. Latar Belakang 3
1.2. Tujuan Praktikum 4
1.3. Manfaat Praktikum 4
BAB III PELAKSANAAN PRAKTIKUM **9**
3.1 Alat dan Bahan 9
3.2 Persiapan Lingkungan Praktikum 9
3.3 Pembuatan Proyek HelloWorld 10
3.4 Implementasi HTML dan CSS 11
3.5 Inisialisasi Repository Git 16
3.6 Pembuatan Repository di GitHub 17
3.7 Menghubungkan Repository Lokal dengan GitHub 18
3.8 Mengunggah Proyek ke GitHub 19
3.9 Praktik Perubahan dan Pembaruan File 20
3.10 Praktik Branch dan Merge 22
3.11 Hasil Akhir Praktikum 24
BAB IV HASIL DAN PEMBAHASAN **26**
4.1 Hasil Implementasi Proyek 26
4.2 Hasil Repository GitHub 26
4.3 Analisis Penggunaan Git dalam Pengelolaan Proyek 26
4.4 Analisis Penggunaan GitHub sebagai Media Penyimpanan dan Kolaborasi 26
4.5 Evaluasi Hasil Praktikum 26
BAB V PENUTUP **27**
DAFTAR PUSTAKA **28**

BAB I
PENDAHULUAN

1.1. Latar Belakang

Pengembangan perangkat lunak pada dasarnya adalah proses yang berlangsung terus-menerus. Kode program tidak pernah ditulis sekali lalu selesai, melainkan terus mengalami penambahan fitur, perbaikan kesalahan, dan penyesuaian tampilan. Oleh karena itu, pengelolaan kode program menjadi aspek penting yang menentukan kualitas, keteraturan, dan keberlanjutan sebuah proyek.

Tanpa alat pengelolaan yang baik, perubahan kode umumnya dikelola secara manual. Cara yang paling sering dijumpai adalah menyalin folder proyek dan memberinya nama berbeda, misalnya "HelloWorld", "HelloWorld-revisi", "HelloWorld-final", dan "HelloWorld-final-fix". Cara ini menimbulkan beberapa permasalahan:

- Sulit mengetahui versi mana yang paling baru dan paling benar.
- Tidak ada catatan yang jelas tentang apa yang berubah, kapan berubah, dan alasan perubahannya.
- Kembali ke kondisi sebelumnya ketika terjadi kesalahan memerlukan pencarian manual yang memakan waktu.
- Penggabungan pekerjaan beberapa orang rentan menimbulkan file yang saling menimpa.
- Risiko kehilangan data meningkat apabila seluruh berkas hanya tersimpan pada satu perangkat.

Untuk mengatasi permasalahan tersebut digunakan sistem kontrol versi (*version control system*). Salah satu sistem yang paling banyak digunakan adalah Git, yaitu sistem kontrol versi terdistribusi yang mencatat riwayat perubahan berkas dari waktu ke waktu. Git dilengkapi layanan hosting seperti GitHub, yang menyediakan tempat penyimpanan repository secara daring sekaligus fitur kolaborasi, sehingga proyek dapat dicadangkan, dibagikan, dan dikerjakan bersama.

Bagi mahasiswa, penguasaan Git dan GitHub merupakan keterampilan dasar yang relevan di dunia pengembangan perangkat lunak, baik untuk tugas perkuliahan, proyek tim, maupun portofolio. Laporan ini disusun berdasarkan praktik langsung mengelola proyek pemrograman web sederhana bernama HelloWorld menggunakan Git dan

GitHub, mulai dari pembuatan repository, pencatatan perubahan, pengunggahan ke GitHub, hingga penggunaan branch dan merge.

1.2. Tujuan Praktikum

Tujuan pelaksanaan praktikum ini adalah sebagai berikut.

1. Memahami konsep dasar Git, GitHub, dan sistem kontrol versi.
2. Mampu menginstal dan mengonfigurasi Git pada komputer.
3. Mampu membuat proyek web sederhana (HelloWorld) menggunakan Visual Studio Code.
4. Mampu menginisialisasi repository Git lokal dan mencatat perubahan menggunakan perintah `git add` dan `git commit`.
5. Mampu membuat repository di GitHub dan menghubungkannya dengan repository lokal.
6. Mampu mengunggah dan memperbarui proyek ke GitHub menggunakan `git push`.
7. Mampu membuat branch baru, melakukan perubahan, dan menggabungkannya ke branch utama menggunakan `git merge`.

1.3. Manfaat Praktikum

Manfaat yang diharapkan dari praktikum ini adalah sebagai berikut.

1. Bagi mahasiswa: memperoleh pengalaman praktik langsung dalam mengelola versi kode program dan memahami alur kerja Git dan GitHub yang digunakan secara luas di industri perangkat lunak.
2. Bagi pengelolaan proyek: memberikan gambaran bahwa riwayat perubahan, pencadangan, dan pengembangan fitur secara terpisah dapat dilakukan secara terstruktur.
3. Bagi pengembangan keterampilan: menjadi dasar untuk mengerjakan proyek tim, berkontribusi pada proyek sumber terbuka, dan membangun portofolio di GitHub.
4. Bagi institusi: menjadi bahan dokumentasi bahwa mahasiswa telah melaksanakan kegiatan praktik sebagai pengganti tugas mata kuliah.

BAB II
LANDASAN TEORI

2.1 Pengertian Git

Git adalah sistem kontrol versi terdistribusi (*distributed version control system*) yang bersifat sumber terbuka dan gratis. Git dirancang untuk mengelola proyek, baik kecil maupun besar, secara cepat dan efisien (Git, n.d.). Git bekerja dengan merekam kondisi berkas proyek pada titik-titik waktu tertentu yang disebut *commit*. Setiap commit menjadi catatan permanen yang dapat dilihat kembali atau dijadikan titik kembali apabila diperlukan.

Ciri utama Git adalah sifatnya yang terdistribusi. Setiap pengguna memiliki salinan lengkap repository, termasuk seluruh riwayat perubahannya, pada komputer masing-masing (Chacon & Straub, 2014). Dengan demikian, sebagian besar operasi Git, seperti membuat commit, melihat riwayat, dan membuat branch, dapat dilakukan tanpa koneksi internet.

2.2 Sejarah dan Perkembangan Git

Git dibuat oleh Linus Torvalds, pencipta kernel Linux, pada tahun 2005. Pembuatannya dilatarbelakangi kebutuhan komunitas pengembang kernel Linux akan sistem kontrol versi baru setelah hubungan dengan alat kontrol versi yang sebelumnya digunakan berakhir (Chacon & Straub, 2014). Tujuan perancangan Git antara lain kecepatan, desain yang sederhana, dukungan kuat untuk pengembangan non-linear melalui ribuan branch paralel, sifat terdistribusi sepenuhnya, serta kemampuan menangani proyek besar seperti kernel Linux secara efisien (Chacon & Straub, 2014).

Sejak saat itu Git berkembang menjadi salah satu sistem kontrol versi yang paling banyak digunakan dalam pengembangan perangkat lunak (Spinellis, 2012). Perkembangan sistem kontrol versi secara umum bergerak dari sistem lokal, ke sistem terpusat, kemudian ke sistem terdistribusi seperti Git (Ruparelia, 2010).

2.3 Pengertian GitHub

GitHub adalah platform berbasis web untuk menyimpan repository Git secara daring (*hosting*) sekaligus menyediakan fitur kolaborasi, seperti pengelolaan isu (*issues*), *pull request*, dan peninjauan kode (GitHub, n.d.). Dengan GitHub, repository lokal dapat

disalin ke server daring sehingga tersedia sebagai cadangan, dapat diakses dari berbagai perangkat, dan dapat dikerjakan bersama orang lain.

2.4 Perbedaan Git dan GitHub

Git dan GitHub sering dianggap sama, padahal keduanya berbeda. Perbedaan utamanya diringkas pada tabel berikut.

Aspek	Git	GitHub
Jenis	Perangkat lunak sistem kontrol versi	Layanan hosting repository berbasis web
Lokasi kerja	Berjalan di komputer lokal	Berjalan secara daring
Fungsi utama	Mencatat dan mengelola riwayat perubahan kode	Menyimpan repository daring dan mendukung kolaborasi
Akses	Melalui terminal atau antarmuka Git	Melalui browser, terminal, atau aplikasi
Ketergantungan	Dapat digunakan tanpa GitHub	Memerlukan Git sebagai dasar pengelolaan repository

Sebagai gambaran, Git dapat dianalogikan sebagai mesin yang mencatat riwayat perubahan, sedangkan GitHub adalah tempat penyimpanan daring bagi hasil catatan tersebut. Selain GitHub, terdapat layanan serupa seperti GitLab dan Bitbucket yang juga memanfaatkan Git.

2.5 Konsep Version Control System

Version Control System (VCS) adalah sistem yang mencatat perubahan pada berkas dari waktu ke waktu sehingga versi tertentu dapat dipanggil kembali di kemudian hari (Chacon & Straub, 2014). Secara umum terdapat tiga jenis VCS.

1. **VCS lokal**: riwayat perubahan disimpan pada satu komputer. Sederhana, tetapi tidak mendukung kerja tim.
2. **VCS terpusat**: riwayat disimpan pada satu server pusat dan pengguna mengambil berkas dari server tersebut. Kelemahannya, server pusat menjadi titik kegagalan tunggal.

3. **VCS terdistribusi**: setiap pengguna memiliki salinan penuh repository beserta riwayatnya. Git termasuk jenis ini, sehingga apabila satu server rusak, repository dapat dipulihkan dari salinan pengguna lain (Chacon & Straub, 2014).

Manfaat VCS antara lain pelacakan riwayat perubahan, kemampuan mengembalikan versi sebelumnya, dukungan kerja paralel melalui branch, dan identifikasi siapa yang mengubah apa.

2.6 Fungsi Repository

Repository (repo) adalah tempat penyimpanan proyek beserta seluruh riwayat perubahannya. Pada Git, repository lokal ditandai dengan adanya folder tersembunyi bernama `.git` di dalam folder proyek. Folder ini menyimpan seluruh data riwayat, sehingga tidak boleh dihapus atau diubah sembarangan.

Fungsi repository meliputi:

- menyimpan berkas proyek dan seluruh riwayat perubahannya;
- menjadi dasar pembuatan branch dan penggabungan;
- menjadi sarana pencadangan dan berbagi proyek, terutama ketika repository disimpan di GitHub (repository jarak jauh atau *remote*);
- menjadi pusat kolaborasi antaranggota tim.

2.7 Pengertian Commit, Push, Pull, Clone, dan Branch

Istilah	Pengertian	Perintah
Commit	Merekam kondisi berkas yang telah dipilih (staged) ke dalam riwayat repository beserta pesan penjelasnya.	`git commit -m "pesan"`
Push	Mengirim commit dari repository lokal ke repository jarak jauh, misalnya GitHub.	`git push`
Pull	Mengambil dan menggabungkan perubahan terbaru dari repository jarak jauh ke repository lokal.	`git pull`
Clone	Menyalin repository jarak jauh beserta seluruh riwayatnya ke komputer lokal.	`git clone <url>`
Branch	Cabang pengembangan yang memungkinkan perubahan dikerjakan terpisah dari jalur utama tanpa mengganggunya.	`git branch`, `git switch`

Selain itu, terdapat istilah *staging area* atau *index*, yaitu area perantara tempat perubahan dikumpulkan melalui `git add` sebelum direkam menjadi commit. Alur kerja dasar Git adalah: mengubah berkas di *working directory*, memilih perubahan ke *staging area*, lalu merekamnya ke repository melalui commit (Chacon & Straub, 2014).

2.8 Peran Git dan GitHub dalam Pengembangan Perangkat Lunak

Dalam pengembangan perangkat lunak modern, Git dan GitHub berperan sebagai berikut.

1. **Pencatatan riwayat**: setiap perubahan tercatat bersama pelaku, waktu, dan alasannya, sehingga mempermudah penelusuran masalah.
2. **Pengembangan paralel**: branch memungkinkan fitur baru atau perbaikan dikerjakan tanpa mengganggu versi yang stabil.
3. **Pencadangan**: repository di GitHub menjadi salinan cadangan di luar komputer lokal.
4. **Kolaborasi tim**: anggota tim dapat bekerja pada proyek yang sama dan menggabungkan hasilnya, antara lain melalui pull request dan peninjauan kode (GitHub, n.d.).
5. **Portofolio dan dokumentasi**: repository publik dapat menjadi bukti pengalaman dan hasil kerja pengembang.

Dengan peran tersebut, Git dan GitHub menjadi keterampilan dasar yang perlu dikuasai mahasiswa yang mempelajari pengembangan perangkat lunak, termasuk pengembangan web.

BAB III
PELAKSANAAN PRAKTIKUM

3.1 Alat dan Bahan

No	Alat/Bahan	Keterangan
1	Laptop atau komputer	MacBook Air dengan chip Apple M4. Sistem operasi: macOS Tahoe 26.2
2	Visual Studio Code	Editor kode untuk menulis file HTML dan CSS.
3	Git	Sistem kontrol versi. Versi: git version 2.50.1 (Apple Git-155)
4	Akun GitHub	Untuk menyimpan repository secara daring. Username: Sahal111
5	Browser	Untuk membuka halaman GitHub dan menampilkan hasil website. Digunakan: Google Chrome
6	Terminal	Untuk menjalankan perintah Git. Digunakan: Terminal bawaan macOS atau terminal terintegrasi Visual Studio Code

3.2 Persiapan Lingkungan Praktikum

a. Instalasi Git

Pada macOS, Git tersedia melalui Xcode Command Line Tools (Apple Git). Apabila Git belum terpasang, perintah `git --version` pada Terminal akan menawarkan pemasangan Command Line Tools. Pemasangan juga dapat dilakukan dengan perintah `xcode-select --install`. Alternatif lain adalah menggunakan Homebrew dengan perintah `brew install git`.

b. Pembuatan akun GitHub

Akun dibuat melalui situs github.com dengan memilih Sign up, mengisi email, kata sandi, dan username, lalu memverifikasi email.

c. Pemeriksaan versi Git.

Setelah instalasi, keberhasilan dipastikan dengan perintah berikut.

```
git –version
```

Perintah ini menampilkan nomor versi Git yang terpasang. Apabila versi tampil, Git siap digunakan. Apabila muncul pesan bahwa perintah tidak dikenali, instalasi atau pengaturan PATH perlu diperiksa. Pada praktikum ini, perintah tersebut menampilkan `git version 2.50.1 (Apple Git-155)`

d. Konfigurasi username dan email.

Git mewajibkan identitas pengguna karena setiap commit mencatat pembuatnya. Identitas diatur sekali dengan perintah berikut.

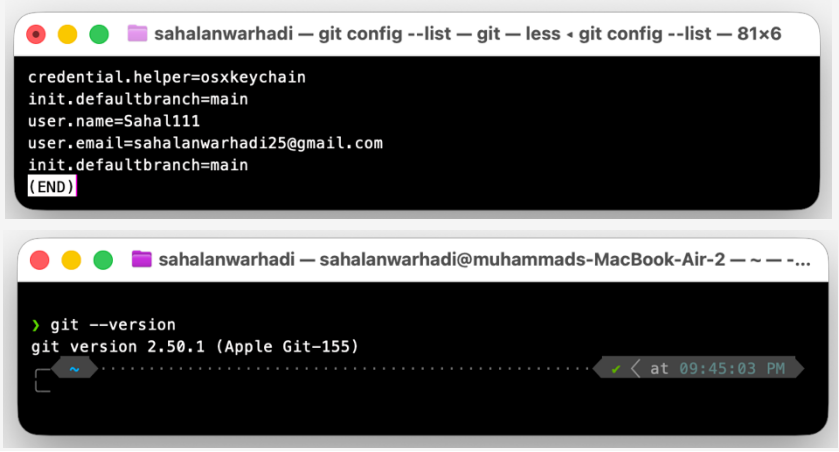
```
git config --global user.name "Sahal111"
git config --global user.email "sahalanwarhadi25@gmail.com"
```

Email sebaiknya sama dengan email yang terdaftar di akun GitHub agar commit terhubung dengan akun tersebut. Konfigurasi diperiksa dengan:

```
git config --list
```

Sebagai pilihan, nama branch awal dapat ditetapkan menjadi `main`:

```
git config --global init.defaultBranch main
```



[GAMBAR 3.1]

Penjelasan: tangkapan layar terminal yang menampilkan versi Git dan hasil `git config --list` *(nama dan email pengguna). Gambar menjadi bukti bahwa Git telah terpasang dan dikonfigurasi.*

3.3 Pembuatan Proyek HelloWorld

Proyek dibuat dengan langkah berikut.

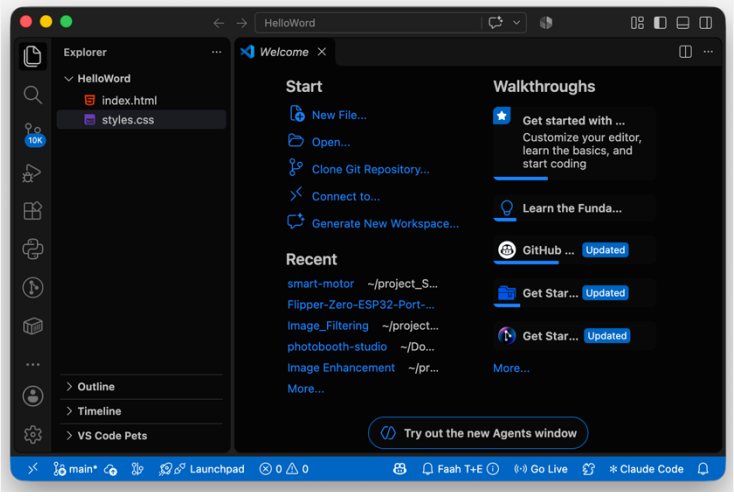
1. Membuat folder baru bernama **HelloWorld** pada lokasi yang mudah diakses.
2. Membuka Visual Studio Code, lalu memilih File > Open Folder dan memilih folder **HelloWorld**.
3. Membuat dua file baru melalui panel Explorer, yaitu **index.html** dan **styles.css**.

Struktur proyek yang dihasilkan:

HelloWorld/

├── index.html

└── styles.css



[GAMBAR 3.2]

Penjelasan: tangkapan layar Visual Studio Code yang memperlihatkan panel Explorer berisi folder HelloWorld dengan file index.html dan styles.css.

3.4 Implementasi HTML dan CSS

a. Struktur file index.html

File ini membentuk kerangka halaman. Berikut contoh isi file sesuai deskripsi proyek

```
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
     scale=1.0" />
    <title>Hello World</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <div class="bola bola1"></div>
    <div class="bola bola2"></div>
    <div class="bola bola3"></div>

    <main class="kaca">
      <h1 class="judul">Hello, World!</h1>
      <p class="sub">Selamat datang di project pertamaku</p>
    </main>
```

```
  </body>
</html>
```

Fungsi tiap bagian kode:

Bagian	Fungsi
`<!DOCTYPE html>`	Mendeklarasikan bahwa dokumen menggunakan HTML5.
`<meta charset>` dan `<meta viewport>`	Mengatur pengodean karakter dan tampilan responsif pada berbagai ukuran layar.
`<title>`	Menentukan judul halaman pada tab browser, yaitu Hello World.
`<link rel="stylesheet">`	Memanggil file styles.css agar tampilan diatur oleh CSS.
`<div class="bola">`	Elemen dekoratif berbentuk bola yang gayanya diatur melalui CSS.
`<main class="kaca">`	Wadah konten utama yang diberi gaya tampilan seperti kaca.
`<h1>` dan `<p>`	Judul "Hello, World!" dan paragraf "Selamat datang di project pertamaku."

b. **Fungsi file styles.css**

File ini mengatur tampilan visual, yaitu warna latar, bentuk dan posisi bola, serta efek kaca pada kotak konten. Berikut isi file.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  font-family: "Segoe UI", Arial, sans-serif;
  background: linear-gradient(-45deg, #0f0c29, #302b63, #24243e,
#1e3a8a);
```

```
  background-size: 400% 400%;
  animation: gradasi 12s ease infinite;
  overflow: hidden;
  position: relative;
}

@keyframes gradasi {
  0%   { background-position: 0% 50%; }
  50%  { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}

/* Bola melayang di latar */
.bola {
  position: absolute;
  border-radius: 50%;
  filter: blur(60px);
  opacity: 0.6;
  animation: melayang 8s ease-in-out infinite;
}

.bola1 {
  width: 300px;
  height: 300px;
  background: #ec4899;
  top: 10%;
  left: 10%;
}

.bola2 {
  width: 350px;
  height: 350px;
  background: #06b6d4;
  bottom: 5%;
  right: 8%;
  animation-delay: -3s;
}

.bola3 {
  width: 200px;
  height: 200px;
```

```
  background: #a855f7;
  top: 55%;
  left: 45%;
  animation-delay: -5s;
}

@keyframes melayang {
  0%, 100% { transform: translateY(0) scale(1); }
  50%      { transform: translateY(-40px) scale(1.1); }
}

/* Kartu kaca */
.kaca {
  position: relative;
  z-index: 1;
  text-align: center;
  padding: 60px 80px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.2);
  border-radius: 24px;
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
  box-shadow: 0 8px 40px rgba(0, 0, 0, 0.35);
  animation: muncul 1.2s ease-out;
}

@keyframes muncul {
  from { opacity: 0; transform: translateY(30px) scale(0.95); }
  to   { opacity: 1; transform: translateY(0) scale(1); }
}

/* Teks bercahaya */
.judul {
  font-size: 4rem;
  font-weight: 800;
  background: linear-gradient(90deg, #f9a8d4, #67e8f9, #c4b5fd,
#f9a8d4);
  background-size: 300% 100%;
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
```

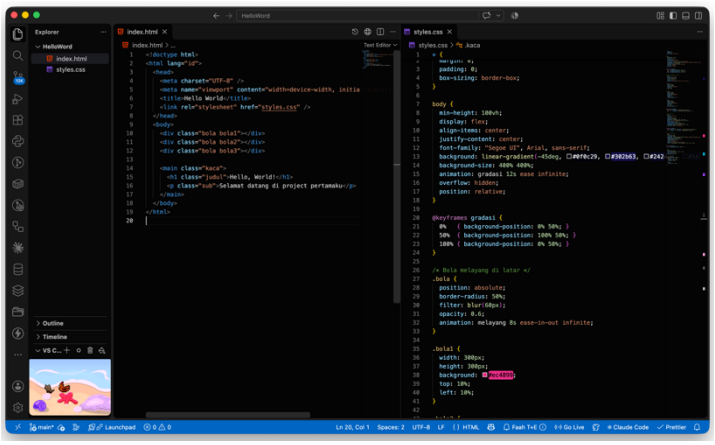
```
  animation: warna 5s linear infinite;
  filter: drop-shadow(0 0 18px rgba(255, 255, 255, 0.35));
}

@keyframes warna {
  to { background-position: 300% 0; }
}

.sub {
  margin-top: 15px;
  color: rgba(255, 255, 255, 0.75);
  font-size: 1.1rem;
  letter-spacing: 2px;
}

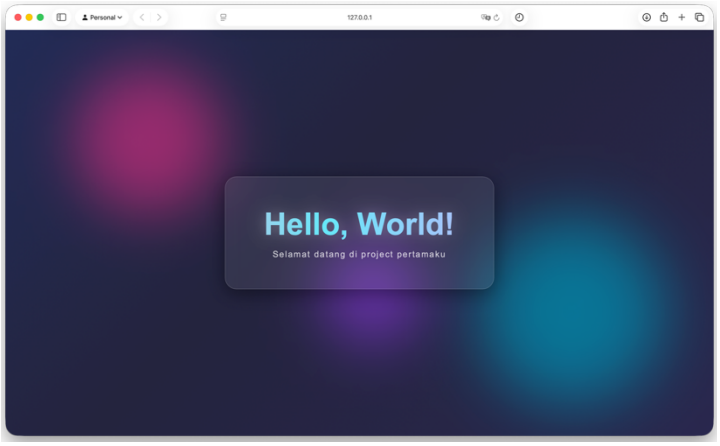
/* Responsif */
@media (max-width: 600px) {
  .kaca {
    padding: 40px 30px;
    margin: 0 20px;
  }
  .judul {
    font-size: 2.4rem;
  }
}
```

`.bola` dan `.kaca` sesuai dengan class pada HTML. Properti `backdrop-filter` memberi efek buram sehingga kotak tampak seperti kaca.



[GAMBAR 3.3]

Penjelasan: tangkapan layar kedua file yang terbuka di editor.



[GAMBAR 3.4]

Penjelasan: hasil membuka index.html di browser, memperlihatkan bola, kotak kaca, judul, dan paragraf.

3.5 Inisialisasi Repository Git

Seluruh perintah dijalankan pada terminal yang berada di dalam folder **HelloWorld**. Terminal dapat dibuka melalui menu Terminal > New Terminal pada Visual Studio Code.

a. git init

`git init` Membuat repository Git lokal dengan membentuk folder tersembunyi `.git`. Setelah perintah ini, Git mulai dapat memantau perubahan di folder tersebut.

b. git status

`git status` Menampilkan kondisi repository, yaitu berkas yang belum dilacak (*untracked*), berkas yang berubah, dan berkas yang siap di-commit. Pada tahap ini, **index.html** dan **styles.css** tampil sebagai berkas yang belum dilacak.

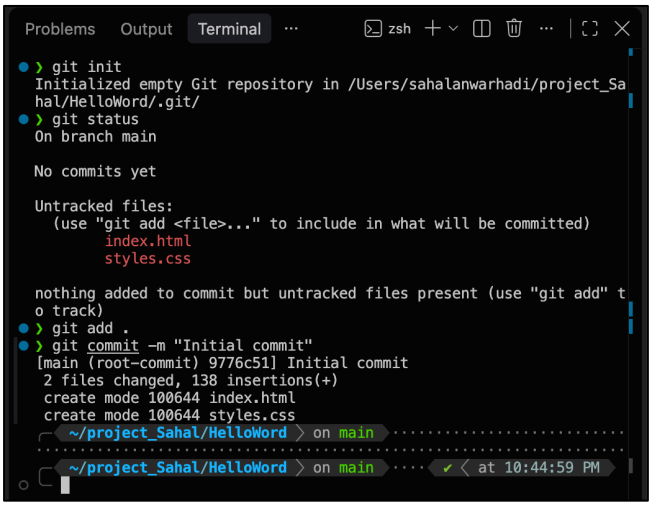
c. git add .

`git add .` Memasukkan seluruh perubahan di folder saat ini ke *staging area*. Tanda titik berarti semua berkas. Setelah itu, `git status` kembali dijalankan untuk memastikan berkas sudah berstatus siap di-commit.

d. git commit

`git commit -m "Initial commit"` Merekam isi *staging area* sebagai commit pertama. Opsi `-m` menambahkan pesan commit. Riwayat dapat diperiksa dengan:

```
git log -oneline
```



[GAMBAR 3.5]

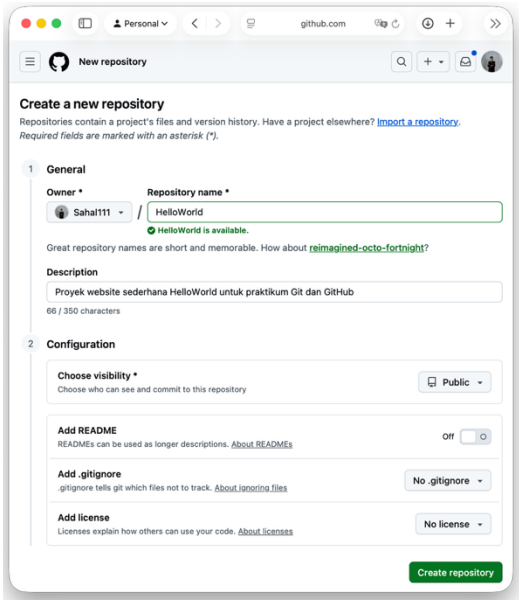
Penjelasan: tangkapan layar rangkaian perintah beserta keluarannya, termasuk status sebelum dan sesudah git add serta konfirmasi commit pertama.

3.6 Pembuatan Repository di GitHub

Langkah pembuatan repository di GitHub adalah sebagai berikut.

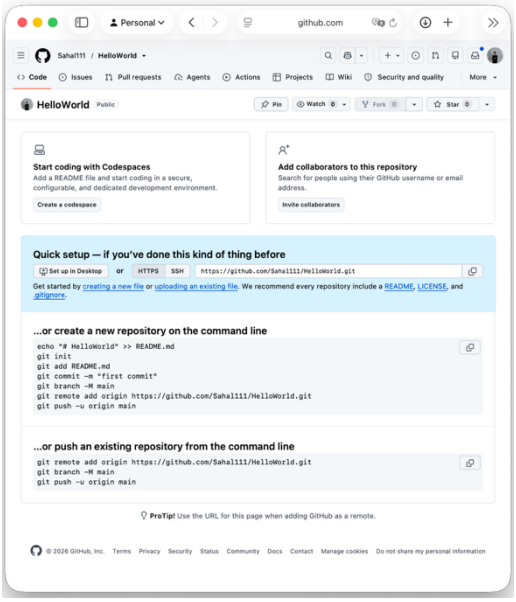
1. Membuka **github.com** dan melakukan login dengan akun yang telah dibuat.
2. Memilih tombol New (atau tanda + di pojok kanan atas, lalu New repository).
3. Mengisi **Repository name** dengan HelloWorld.
4. Mengisi **Description** dengan keterangan singkat, misalnya "Proyek website sederhana HelloWorld untuk praktikum Git dan GitHub".
5. Menentukan visibilitas: **Public** (dapat dilihat siapa saja) atau **Private** (hanya dapat dilihat pemilik dan pihak yang diberi akses). Pilihan yang digunakan pada praktikum ini Public. Apabila laporan akan diperiksa dosen melalui tautan, pilih Public atau berikan akses kepada dosen.
6. Tidak mencentang opsi penambahan README, .gitignore, atau license, karena repository lokal sudah memiliki commit. Opsi tersebut dapat menimbulkan konflik riwayat saat push pertama.
7. Memilih **Create repository**.

Setelah dibuat, GitHub menampilkan halaman berisi alamat repository (URL) yang akan digunakan pada tahap berikutnya.



[GAMBAR 3.6]

Penjelasan: tangkapan layar formulir pembuatan repository yang telah terisi nama, deskripsi, dan pilihan Public atau Private.



[GAMBAR 3.7]

Penjelasan: tangkapan layar halaman repository sesudah dibuat, memperlihatkan URL repository dan petunjuk menghubungkan repository lokal.

3.7 Menghubungkan Repository Lokal dengan GitHub

Pada tahap ini repository lokal dihubungkan dengan repository kosong di GitHub

a. git remote add origin

```
git remote add origin https://github.com/Sahal111/HelloWorld.git
```

Menambahkan repository jarak jauh dengan nama `origin` yang menunjuk ke URL repository GitHub. Nama `origin` adalah konvensi untuk repository jarak jauh utama. Hasilnya dapat diperiksa dengan:

```
git remote -v
```

b. git branch -M main

```
git branch -M main
```

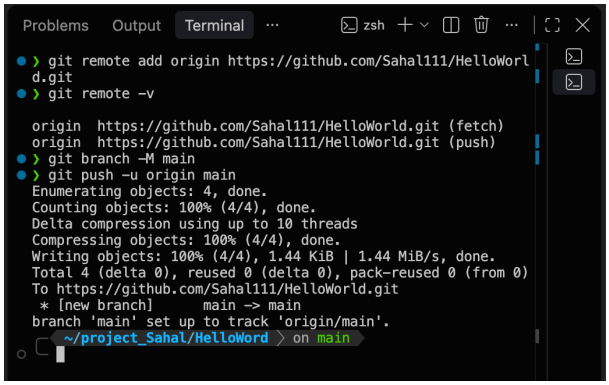
Mengganti nama branch aktif menjadi `main`. Opsi `-M` memaksa penggantian nama walaupun nama tersebut sudah ada. Langkah ini menyamakan nama branch lokal dengan branch utama pada GitHub.

c. git push -u origin main

```
git push -u origin main
```

Mengirim branch `main` ke repository `origin`. Opsi `-u` (*set upstream*) menghubungkan branch lokal `main` dengan `origin/main`, sehingga perintah berikutnya cukup `git push` atau `git pull` tanpa menyebut tujuan.

Pada push pertama, Git dapat meminta autentikasi ke GitHub. GitHub tidak lagi menerima kata sandi akun untuk operasi Git melalui HTTPS, sehingga autentikasi dilakukan melalui jendela login browser (Git Credential Manager), *personal access token*, atau SSH (GitHub, n.d.).



[GAMBAR 3.8]

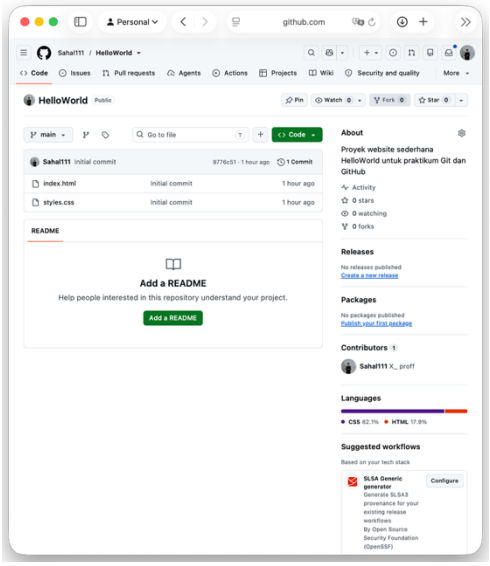
Penjelasan: tangkapan layar yang memperlihatkan perintah dan keluaran push, termasuk konfirmasi bahwa branch main terhubung dengan origin/main.

3.8 Mengunggah Proyek ke GitHub

Proses unggah dilakukan melalui `git push -u origin main` pada tahap 3.7. Untuk memastikan hasilnya, dilakukan pemeriksaan berikut.

1. Memuat ulang (*refresh*) halaman repository HelloWorld di GitHub.
2. Memeriksa bahwa file **index.html** dan **styles.css** tampil pada daftar berkas.
3. Memeriksa bahwa commit "Initial commit" tampil pada riwayat commit beserta nama pembuat dan waktunya.
4. Membuka masing-masing file untuk memastikan isinya sama dengan yang ada di komputer lokal.

5. Menjalankan `git status` pada terminal. Apabila berhasil, Git melaporkan bahwa branch `main` mutakhir terhadap `origin/main` dan tidak ada perubahan yang tertunda.



[GAMBAR 3.9]

*Penjelasan: tangkapan layar halaman repository yang menampilkan **index.html**, **styles.css**, dan commit "Initial commit".*

3.9 Praktik Perubahan dan Pembaruan File

Tahap ini mensimulasikan alur kerja pengembangan yang umum, yaitu mengubah file, memeriksa perubahan, lalu mencatat dan mengirimnya.

a. Mengubah file.

Sebagai perubahan sederhana, isi paragraf pada **index.html** diubah. Contoh perubahan:

```
<!-- Sebelum -->
<p class=”sub”>Selamat datang di project pertamaku.</p>

<!-- Sesudah -->
<p class=”sub”>Selamat datang di project pertamaku. Proyek ini
dikelola dengan Git dan GitHub.</p>
```

Setelah file disimpan (Cmd + S), perubahan tampil di Visual Studio Code dan dapat dilihat di browser.

b. Memeriksa perubahan dengan git status.

```
git status
```

Git melaporkan bahwa **index.html** telah dimodifikasi (*modified*) tetapi belum di-*stage*.

c. Melihat perubahan dengan git diff.

```
git diff
```

Menampilkan perbedaan baris demi baris antara berkas di *working directory* dan versi terakhir yang di-commit. Baris yang dihapus ditandai `-` dan baris yang ditambahkan ditandai `+`. Setelah perubahan di-*stage*, perbedaan dilihat dengan `git diff --staged`.

d. Menambahkan perubahan.

```
git add index.html
```

Memasukkan perubahan pada **index.html** ke *staging area*. Berbeda dengan `git add .`, perintah ini hanya memilih berkas tertentu.

e. Melakukan commit baru.

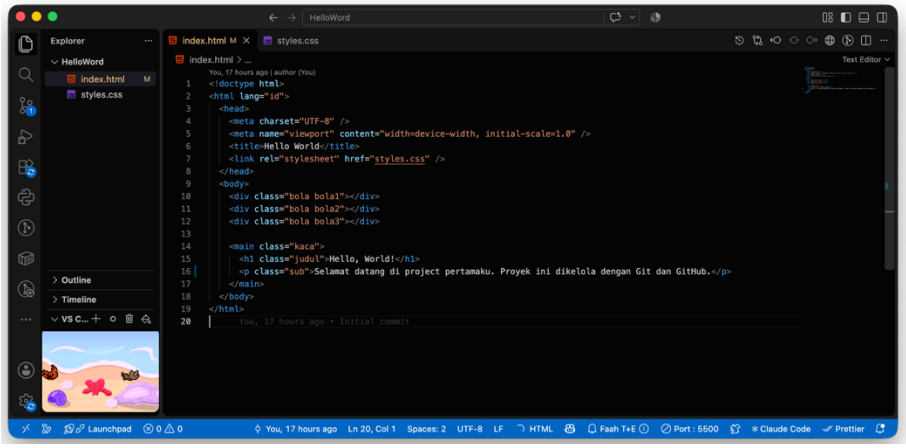
```
git commit -m "Memperbarui teks paragraf pada halaman utama"
```

Pesan commit sebaiknya singkat dan menjelaskan isi perubahan.

f. Mengirim perubahan dengan git push.

```
git push
```

Karena branch sudah terhubung ke `origin/main` melalui opsi `-u`, perintah cukup `git push`. Setelah itu, halaman GitHub dimuat ulang untuk memastikan commit kedua dan isi file terbaru tampil.



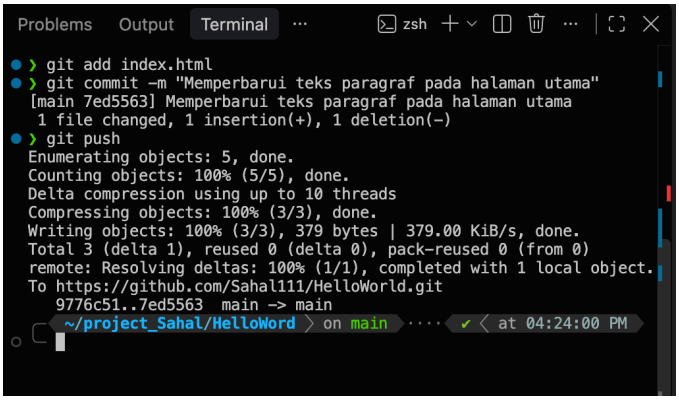
[GAMBAR 3.10]

Penjelasan: tangkapan layar editor yang memperlihatkan perubahan teks paragraf, dapat berupa tampilan penanda perubahan atau diff bawaan editor.



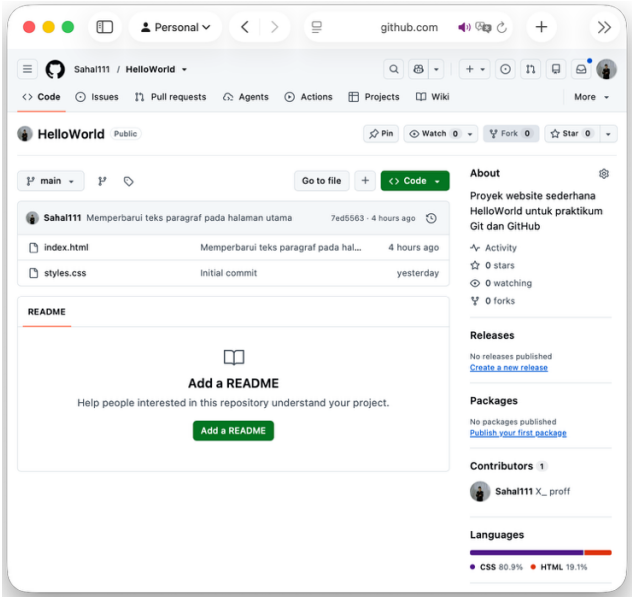
[GAMBAR 3.11]

Penjelasan: tangkapan layar status modified dan perbedaan baris yang ditampilkan oleh git diff.



[GAMBAR 3.12]

Penjelasan: tangkapan layar commit kedua dan proses push ke GitHub.



[GAMBAR 3.13]

Penjelasan: tangkapan layar halaman Commits di GitHub yang menampilkan dua commit.

3.10 Praktik Branch dan Merge

Branch digunakan untuk mengerjakan perubahan tanpa mengganggu branch `main`. Pada praktik ini dibuat branch `fitur-warna` untuk mengubah warna bola pada **styles.css**.

a. Melihat branch yang ada.

```
git branch
```

Branch aktif ditandai dengan tanda bintang (*).

b. Membuat dan berpindah ke branch baru.

```
git switch -c fitur-warna
```

Perintah ini membuat branch `fitur-warna` sekaligus berpindah ke branch tersebut. Perintah setara pada versi Git yang lebih lama adalah `git checkout -b fitur-warna`. Untuk berpindah antar branch digunakan `git switch <nama-branch>`.

c. Mengubah kode.

Pada **styles.css**, warna gradien `.bola` diubah, misalnya:

```
/* Sebelum */
background: linear-gradient(90deg, #f9a8d4, #67e8f9, #c4b5fd,
#f9a8d4);

/* Sesudah */
background: linear-gradient(135deg, #a1ffce, #faffd1);
```

d. Melakukan commit pada branch.

```
git add styles.css
git commit -m "Mengubah warna bola pada styles.css"
```

Pada tahap ini, perubahan hanya ada di branch `fitur-warna`. Apabila berpindah kembali ke `main` dengan `git switch main`, tampilan bola kembali ke warna semula.

e. Menggabungkan ke branch main.

```
git switch main
git merge fitur-warna
```

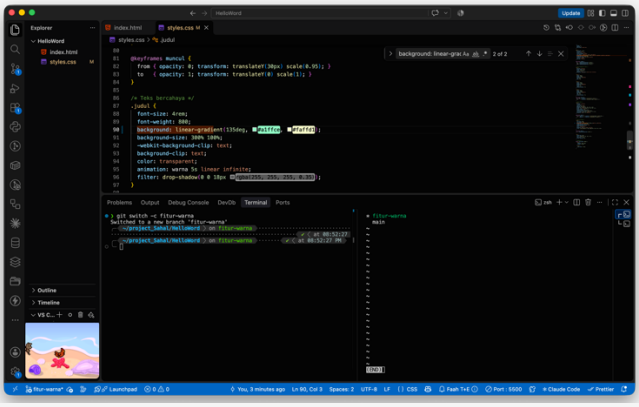
Perintah `git merge` menggabungkan riwayat `fitur-warna` ke branch yang sedang aktif, yaitu `main`. Apabila `main` tidak memiliki perubahan baru selama branch dikerjakan, Git melakukan *fast-forward merge* tanpa commit penggabungan tambahan. Apabila kedua branch mengubah baris yang sama, terjadi konflik yang harus diselesaikan manual sebelum commit penggabungan.

f. Mengirim hasil dan merapikan branch.

```
git push
git branch -d fitur-warna
```

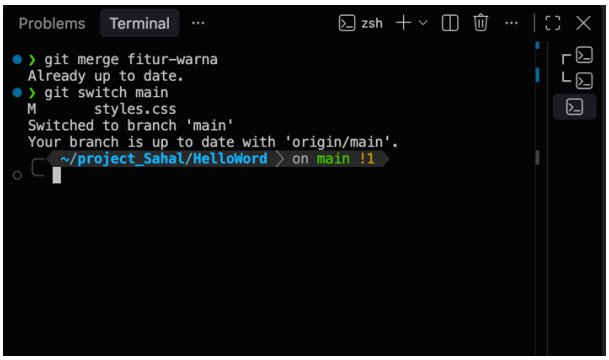
Perintah pertama mengirim `main` yang telah digabung ke GitHub. Perintah kedua menghapus branch lokal `fitur-warna` yang sudah digabung. Penghapusan bersifat opsional. Riwayat dapat divisualisasikan dengan:

```
git log --oneline --graph –all
```



[GAMBAR 3.14]

*Penjelasan: tangkapan layar perintah git switch -c, git branch, dan perubahan kode pada **styles.css**.*



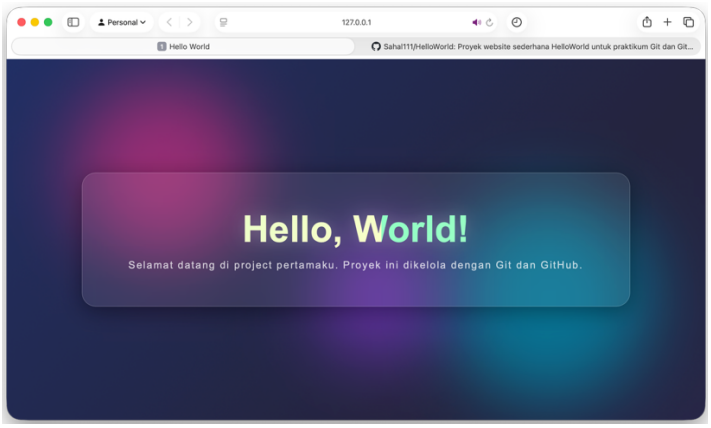
[GAMBAR 3.15]

Penjelasan: tangkapan layar keluaran git merge yang menunjukkan penggabungan berhasil.

3.11 Hasil Akhir Praktikum

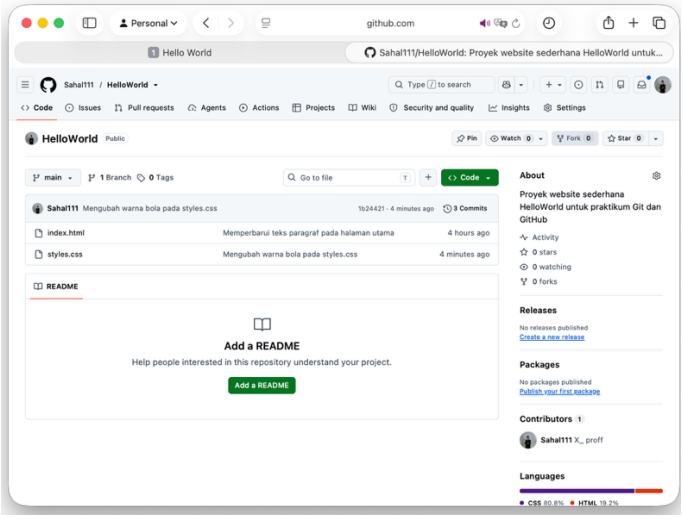
Pada akhir praktikum, proyek HelloWorld telah berada dalam kondisi berikut:

- repository lokal terinisialisasi Git dan memiliki beberapa commit: commit awal, commit pembaruan teks, dan commit perubahan warna;
- repository di GitHub terhubung dengan repository lokal melalui remote `origin`;
- file **index.html** dan **styles.css** tersimpan di GitHub dalam versi terbaru;
- perubahan dari branch `fitur-warna` telah digabung ke branch `main`;
- riwayat perubahan dapat ditelusuri baik melalui `git log` maupun halaman Commits di GitHub.



[GAMBAR 3.17]

Penjelasan: tangkapan layar halaman website setelah seluruh perubahan.



[GAMBAR 3.18]

Penjelasan: tangkapan layar halaman utama repository setelah seluruh perubahan di-push.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil Implementasi Proyek

Proyek HelloWorld terdiri atas `index.html` yang memuat struktur halaman (judul, elemen `bola`, elemen `kaca`, heading, dan paragraf) serta `styles.css` yang mengatur tampilan visualnya. Saat dibuka di browser, halaman tampil sesuai kode tersebut (Gambar 3.4 dan Gambar 3.17). Proyek sengaja dibuat sederhana agar fokus praktikum tertuju pada pengelolaan versi.

4.2 Hasil Repository GitHub

Setelah push, repository GitHub memuat file `index.html` dan `styles.css` besert a riwayat commit, lengkap dengan pembuat, waktu, dan pesan commit (Gambar 3.9, 3.13, dan 3.18). Repository yang digunakan: https://github.com/Sahal111/HelloWorld d engan visibilitas [Public].

4.3 Analisis Penggunaan Git dalam Pengelolaan Proyek

Dibandingkan menyalin folder secara manual, Git mencatat setiap perubahan sebagai commit yang dapat ditelusuri. Perintah `git status` dan `git diff` membantu memeriksa perubahan sebelum dicatat, sedangkan `git add` memungkinkan memilih perubahan yang masuk ke commit. Branch `fitur-warna` memungkinkan perubahan warna dikerjakan terpisah dari `main`sampai digabung dengan `git merge`. Keterbatasannya, perintah terminal memerlukan pembiasaan, dan karena proyek dikerjakan sendiri, konflik penggabungan antar pengembang tidak teruji.

4.4 Analisis Penggunaan GitHub sebagai Media Penyimpanan dan Kolaborasi

GitHub berfungsi sebagai cadangan di luar komputer lokal, dapat diakses dari perangkat lain melalui `git clone`, dan memudahkan pihak lain melihat hasil kerja serta riwayat commit. Fitur kolaborasi seperti pull request dan issues (GitHub, n.d.) tidak diuji karena proyek dikerjakan sendiri.

4.5 Evaluasi Hasil Praktikum

Praktikum mencakup alur inti Git dan GitHub: konfigurasi, inisialisasi repository, commit, push, pembaruan file, serta branch dan merge. Seluruh tahapan dibuktikan dengan tangkapan layar pada Bab III. Hal yang belum dicakup adalah penanganan konflik penggabungan, pull request, dan `.gitignore`, yang dapat menjadi materi praktik lanjutan.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum pengelolaan proyek HelloWorld menggunakan Git dan GitHub, dapat disimpulkan sebagai berikut.

1. Git adalah sistem kontrol versi yang berjalan di komputer lokal untuk mencatat riwayat perubahan kode, sedangkan GitHub adalah layanan penyimpanan repository secara daring. Keduanya saling melengkapi.
2. Proyek HelloWorld berhasil dikelola menggunakan Git melalui perintah `git init`, `git status`, `git add`, dan `git commit`.
3. Repository lokal berhasil dihubungkan dengan GitHub melalui `git remote add origin`, lalu dikirim dan diperbarui menggunakan `git push`.
4. Branch dan merge memungkinkan perubahan dikerjakan terpisah dari `main` tanpa mengganggu versi utama proyek.
5. Dibandingkan menyalin folder secara manual, Git dan GitHub membuat riwayat perubahan lebih jelas, versi lebih mudah ditelusuri, dan proyek lebih aman karena tersimpan di luar komputer lokal.

DAFTAR PUSTAKA

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

Git. (n.d.). *Git documentation*. Diakses 3 November 2026, dari https://git-scm.com/doc

GitHub. (n.d.). *GitHub Docs*. Diakses 3 November 2026, dari https://docs.github.com

Ruparelia, N. B. (2010). The history of version control. *ACM SIGSOFT Software Engineering Notes, 35*(1), 5–9. https://doi.org/10.1145/1668862.1668876

Spinellis, D. (2012). Git. *IEEE Software, 29*(3), 100–101. https://doi.org/10.1109/MS.2012.61